

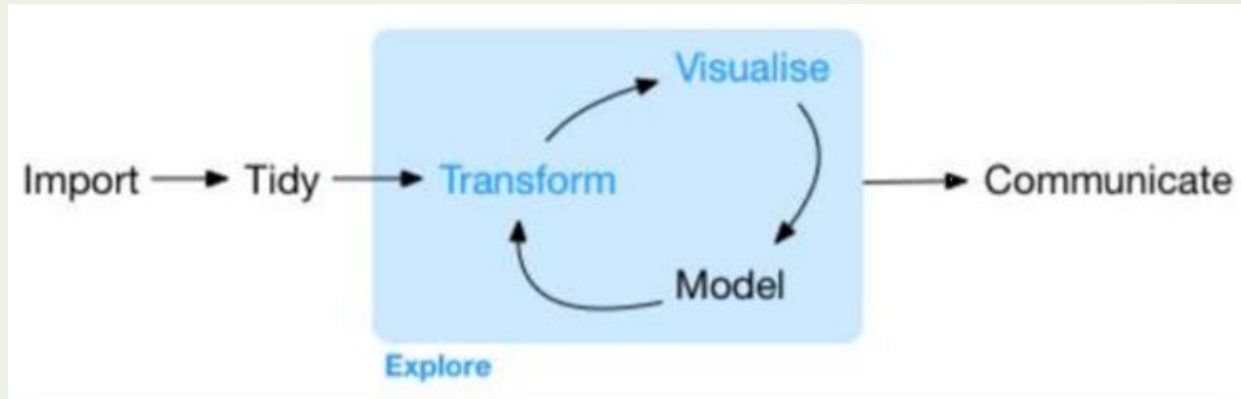
Mini curso



Fernanda Abduche; Gilberto Rodrigues Liska; Josiane Rodrigues.

Introdução

Fluxo de trabalho para ciência de dados



Fonte: Ciência de dados com R - introdução



O que é e por que aprender o R?

O R é uma linguagem de programação estatística, conjunto de pacotes e ferramentas. Para criação de simples rotinas até análises complexas.

- ❖ É excelente para criar rotinas e sistematizar tarefas repetitivas;
- ❖ Quantidade enorme de pacotes, para diversos tipos de necessidades;
- ❖ Ótima ferramenta para criar relatórios e gráficos.

RStudio

File Edit Code View Plots Session Build Debug Profile Tools Help

Go to file/function Addins Project: (None)

1

1

Source on Save Run Source

1:1 (Top Level) R Script

Console Terminal Jobs

Digite 'demo()' para demonstrações, 'help()' para o sistema on-line de ajuda, ou 'help.start()' para abrir o sistema de ajuda em HTML no seu navegador. Digite 'q()' para sair do R.

2

>

Environment History Connections

Import Dataset

Global Environment

Environment is empty

3

Files Plots Packages Help Viewer

New Folder Delete Rename More

Home

	Name	Size	Modified
☐	.Rhistory	250 B	May 4, 2020, 7:3
☐	Meus Vídeos		
☐	Minhas Músicas		
☐	Modelos Personalizados do Office		
☐	R		
☐	Zoom		

4



ITEM 1: Editor de código: escreve e edita os scripts. Script é uma sequência de comandos que serão executados em sequência pelo R. (File -> New File -> Rscript)

Ao escrever o comando para obter o resultado do mesmo no console basta apertar o ícone RUN ou utilizar o atalho: ctrl + enter. Prestar atenção se este comando não é dependente de outro.

Salvando scripts: No editor de código é possível salvar os scripts para continuar em outro momento. Para salvar: File e Save, ou utilize o atalho: ctrl + S.

Comentários do código: Para deixar um comentário utilize o símbolo #, este então não irá interferir nos comandos e ficará de cor diferente.

ITEM 2: Console: a maioria dos resultados vai aparecer no console. É onde você pode digitar suas ordens (comandos) e também é onde o R vai “responder”.

Este símbolo, **>**, indica a linha onde você deve inserir os comandos. Digite e aperte enter. Por exemplo, digite um dos comandos:

#Exemplo 1:

$7 * 9 + 2 * 6$

$2.5 * 4$

$(50 + 7)/(8 * (3 - 5/2))$

$3 ^ 4$

ITEM 3: Environment e History: No Environment ficarão guardados todos os objetos que forem criados na sessão do R. E na aba History o RStudio cria um histórico de comandos utilizados.

ITEM 4:

- ❖ **Files:** Onde é definido o diretório dos arquivos e assim os mesmos podem ser localizados no script.
- ❖ **Plots:** A aba Plots trará os gráficos gerados, possibilitando a exportação para alguns formatos diferentes, como .png e .pdf.
- ❖ **Packages:** Em Packages estão listados os pacotes instalados.
- ❖ **Help:** É a aba para pedir ajuda. Quando deseja saber a função de algum código basta digitar no editor **?nomedafunção** e então na aba help será dada toda a descrição da função.
- ❖ **Viewer:** Visualizar conteúdo da web.

Objetos (variáveis): os objetos no R correspondem a uma estrutura que irá receber algum valor. Para criar objetos deve-se usar o símbolo `<-`. Todos os objetos criados ficam arquivados na aba Environment.

#Exemplo 2:

```
x <- 15  
x + 5  
x * x / 2  
2 ^ x  
y <- x / 3
```

Funções: O programa já vem configurado com diversas funções e outras podem ser instaladas usando os “pacotes”.

Uma função tem dois elementos básicos, o **nome** e os **parâmetros**, por exemplo: `log(10)` é uma função que possui um nome e um parâmetro que é o número.

Algumas funções:

Função R	Estatística
<code>sum()</code>	Soma de valores
<code>mean()</code>	Média
<code>var()</code>	Variância
<code>median()</code>	Mediana
<code>summary()</code>	Resumo estatístico
<code>quantile()</code>	Quantis

Exemplo 3:

```
vetor.num <- c(1, 2, 5, 8, 1001)
```

```
1+2+5+8+1001
```

```
sum(vetor.num)
```

```
(1+2+5+8+1001)/5
```

```
mean(vetor.num)
```

OBS.: Da mesma forma pode-se usar as outras funções.



Pacotes: São conjuntos extras de funções do R que podem ser instalados.

Antes de iniciar um código novo é preciso instalar um pacote usando o comando:

install.packages(“escrever o nome do pacote”)

Em seguida para usar o comando devemos usar:

library(nome do pacote)



Exemplo 4:

```
vetor.num <- c(1, 2, 5, 8, 1001)  
basicStats(vetor.num)  
install.packages("fBasics")  
library(fBasics)  
basicStats(vetor.num)
```

Definindo o local dos Dados

Como direcionar o R para ler o arquivo que deseja trabalhar, para informar ao R qual pasta ele deve ler, utilizar o comando:

setwd(colocar aqui o link da pasta do arquivo).

Após isso é necessário fazer a leitura do arquivo. O R base possui funções para a leitura dos principais tipos de arquivos. Um outro pacote específico, e muito bom para isso, é o **readr**.

#Exemplo 5:

```
setwd("C:/Users/Fernanda Abduche/Desktop/facul/grupo de estudo em estatistica")  
dados<-read.table("Tab1.txt", head=T)
```



O **head** e o **tail** são funções para ver a “cabeça” e o “rabo” dos seus dados, ou seja, o começo e o fim das amostras. Essa observação ajuda a identificar erros básicos no carregamento, possibilitando ajustes o quanto antes.

Outros comandos muito importantes para começar a investigar os dados são o **str()**, o **class()** e o **summary()**.

Para verificar o tipo do objeto, ou seja, sua classe, utilize: **class(dados)**.

Já para verificar a estrutura do objeto, ou seja, seus campos (quando aplicável), insira: **str(dados)**.

Para verificar estatísticas básicas do objeto (média, mediana, quantis, mínimo, máximo etc.), quando aplicáveis: **summary(dados)**.

Exemplo 6:

head(dados)

tail(dados)

class(dados)

str(dados)

summary(dados)

head(dados\$idade)

head(dados\$Idade)

tail(dados\$Idade)

class(dados\$Idade) # Informa o tipo da coluna



Manipulando os dados

A manipulação dos dados é como ato de transformar, limpar, agregar e juntar os dados.

Existem diversos tipos de objetos, e cada tipo “armazena” um conteúdo diferente, desde tabelas de dados recém-carregados a textos, números, ou simplesmente a afirmação de verdadeiro ou falso.



Usaremos a função **class()** para ver o tipo de cada uma.

Numeric para valores inteiros ou decimais,
Character para valores textuais,
Logical para valores lógicos (verdadeiro ou falso).

Exemplo 7:

```
inteiro <- 928  
outro.inteiro <- 5e2  
decimal <- 182.93  
caracter <- 'exportação'  
logico <- TRUE  
outro.logico <- FALSE
```

```
class(inteiro)  
class(logico)
```



Além dos tipos básicos, existem também os tipos “complexos”, que são vetor, matriz, lista, data.frame e fator.

A função `matrix` cria uma matriz com os valores do argumento `data`. O números de linhas e colunas são definidos pelos argumentos **`nrow`** e **`ncol`**.

Exemplo 8:

```
minha.matriz <- matrix(data=1:12,nrow=3,ncol=4)  
minha.matriz
```

Data.frame é o mais utilizado, nele é armazenado conjunto de dados estruturados em linhas e colunas.

Vetor também é muito usado e nele estão sequências unidimensionais de valores de um mesmo tipo, podendo ser feitas as seguintes atribuições:

Exemplo 9:

```
#faça as seguintes atribuições  
vetor.chr <- c('tipo1', 'tipo2', 'tipo3', 'tipo4')  
vetor.num <- c(1, 2, 5, 8, 1001)  
vetor.num.repêtidos <- c(rep(2, 50)) #usando função para repetir  
números vetor.num.sequencia <- c(seq(from=0, to=100, by=5))  
#usando função para criar sequências  
vetor.logical <- c(TRUE, TRUE, TRUE, FALSE, FALSE)
```



Conhecendo o data.frame e o vetor, você será capaz de entender como os dois se relacionam. Cada coluna de um data frame é um vetor.

Como criar um data.frame, exemplo 10:

```
#cria-se diferentes vetores  
nome <- c('João', 'José', 'Maria', 'Joana')  
idade <- c(45, 12, 28, 31)  
adulto <- c(TRUE, FALSE, TRUE, TRUE)  
uf <- c('DF', 'SP', 'RJ', 'MG')  
  
#cada vetor é uma combinação de elementos de um MESMO tipo de dados  
#sendo assim, cada vetor pode ser uma coluna de um data.frame  
  
clientes <- data.frame(nome, idade, adulto, uf)  
clientes
```

Para selecionar ou trabalhar separadamente com apenas um campo (coluna) do seu data.frame, deve-se utilizar o \$.

Usando termos mais técnicos, um data frame é um conjunto de dados HETEROGÊNEOS, pois cada coluna pode ser de um tipo, e BIDIMENSIONAL, por possuir apenas linhas e colunas. Já o vetor é um conjunto de dados HOMOGÊNEO, pois só pode ter valores de um mesmo tipo, e UNIDIMENSIONAL.

Uma notação parecida é usada para selecionar elementos no data.frame. Porém, como já comentamos, data frames são BIDIMENSIONAIS. Então usaremos a notação [,] com uma vírgula separando qual a linha (posição antes da vírgula) e a coluna (posição após a vírgula) que queremos selecionar.

Exemplo 11:

```
clientes$nome  
clientes$uf
```

```
clientes[1,1]  
clientes[,1]  
clientes[1,]  
clientes[,2:3]
```

Valores faltantes e o “NA”

Em casos onde não existe valor em uma coluna de uma linha, o R atribui NA. Ou no caso de valores omissos os mesmos deverão ser substituídos por NA.

Seguem algumas funções úteis para lidar-se com NA:

- ❖ A função **summary()** pode ser usada para averiguar a ocorrência de NA.
- ❖ A função **is.na()** realiza um teste para saber se a variável/coluna possui um valor NA. retorna TRUE se for NA e FALSE se não for.
- ❖ A função **complete.cases()** retorna TRUE para as linhas em que todas as colunas possuem valores válidos (preenchidos) e FALSE para as linhas em que, em alguma coluna, existe um NA. Ou seja, esta função diz quais são as linhas (amostras) completas em todas as suas características (campos).
- ❖ Algumas funções possuem o argumento **na.rm**, ou semelhantes, para desconsiderar NA no cálculo. É o caso da função **mean()** ou **sum()**.

Exemplo 12.A:

```
x <- c(1,2,NA,3)
x
mean(x)
mean(x, na.rm=TRUE)

xn=na.omit(x)
xn
mean(xn)
```

Exemplo 12.B:

```
DF <- data.frame(x = c(1, 2, 3), y =
c(0, 10, NA))
DF
na.omit(DF)
```

Estrutura de controle de fluxo

A estrutura de **if** (se) e **else** (então) usa os operadores lógicos apresentados anteriormente.

Se a condição for verdadeira, **if**, opera uma função. Mas se for falsa opera outra função. Da mesma forma, existe uma função que gera o mesmo resultado, o **ifelse()**.

A estrutura **if() {} else {}** só opera variáveis, uma por uma, já a estrutura **ifelse()** opera vetores, ou seja, consegue fazer a comparação para todos os elementos. Isso faz com que a forma **if() {} else {}** seja mais utilizada para comparações fora dos dados, com variáveis avulsas. Já a estrutura **ifelse()** é mais usada para comparações dentro dos dados, com colunas, vetores e linhas.



Em todas as comparações do R usamos operadores lógicos. São operações matemáticas em que o resultado é TRUE ou FALSE (tipo logic). Alguns exemplos:

- ❖ == igual a: compara dois objetos e se forem iguais retorna TRUE, caso contrário, FALSE;
- ❖ != diferente: compara dois objetos e se forem diferentes retorna TRUE, caso contrário, FALSE;
- ❖ | ou (or): compara dois objetos, se um dos dois for TRUE, retorna TRUE, se os dois forem FALSE, retorna FALSE;
- ❖ & e (and): compara dois objetos, se os dois forem TRUE, retorna TRUE, se um dos dois ou os dois forem FALSE, retorna FALSE;
- ❖ >, >=, <= maior, maior ou igual, menor, menor ou igual: compara grandeza de dois números e retorna TRUE ou FALSE conforme a condição;

Exemplos 13.A:

```
a <- 9823  
if(a >= 10000) {  
  b <- 'VALOR ALTO'  
} else if(a < 10000 & a >= 1000) {  
  b <- 'VALOR MEDIO'  
} else if(a < 1000) {  
  b <- 'VALOR BAIXO' }  
b
```

Exemplo 13.B:

```
a <- 839  
c <- ifelse(a >= 10000,  
'VALOR ALTO', ifelse(a <  
10000 & a >= 1000, 'VALOR  
MEDIO', 'VALOR BAIXO'))  
c
```



Loops: repetem uma sequência de comandos quantas vezes você desejar, ou até que uma condição aconteça, variando-se alguns aspectos entre uma repetição e outra.

For: é usado para realizar uma série de ordens para uma determinada sequência ou índices (vetor).

Para cada valor (chamamos esse valor de *i*) dentro do vetor **c(1, 2, 3, 4, 5)**, execute o comando **print(i^2)**. Qualquer outro comando dentro das chaves { ... } seria executado para cada valor do vetor.

Exemplo 14:

```
for(i in c(1, 2, 3, 4, 5)) {  
  print(i^2)  
}
```

Também é possível utilizar loop com if. No exemplo a seguir, queremos ver todos os números de 1 a 1000 que são divisíveis por 29 e por 3 ao mesmo tempo. Para isso, utilizaremos o **operador %%**, que mostra o resto da divisão. Se o resto for zero, é divisível. Exemplo 15:

```
15%%3
15%%4
for(i in 1:1000){
  if((i %% 29 == 0) & (i %% 3 == 0)){
    print(i)
  }
}
```

While: é uma estrutura de controle de fluxo do tipo loop, mas, diferentemente do for(), o while executa as tarefas repetidamente até que uma condição seja satisfeita, não percorrendo um vetor. Exemplo 16:

```
i <- 1
while(i <= 5){
  print(i)
  i <- i + 1
}
```

Funções

Funções “encapsulam” uma sequência de comandos e instruções. É uma estrutura nomeada, que recebe parâmetros para iniciar sua execução e retorna um resultado ao final.

```
sua_funcao <- function(parametro1, parametro2){  
# sequência de tarefas  
return(valores_retornados) }  
# chamada da função  
sua_função
```

Exemplo 17:

**#O sistema de avaliação de uma disciplina em uma
#universidade consiste na realização de três provas
#e um trabalho.**

**#A Nota Média do aluno é calculada pela seguinte expressão
#Nota Média = $(P1 + P2 + P3 + T)/4$**

**#Se a Nota Média do aluno for no mínimo 6 pontos,
#ele é aprovado, caso contrário é reprovado.
#Considerando um aluno que obteve as seguintes notas:
#P1=5, P2=3, P3=5, Trabalho=6, construa uma rotina que calcule
#a Nota Média e informe a situação do aluno.**

Exemplo 17:

```
nm=function(p1,p2,p3,t){  
  n=(p1+p2+p3+t)/4  
  
  Situacao=ifelse(n>=6,"Aprovado", "Reprovado")  
  return(list(Nota=n,"Situação"=Situacao))  
}  
nm(5,3,5,6)  
nm(10,10,10,10)
```



Pacote dplyr

Ao carregar o **pacote tidyverse**, você já carregará automaticamente o **pacote dplyr**. O forte do pacote dplyr é a sintaxe simples e concisa, o que facilita o aprendizado e torna o pacote um dos preferidos para as tarefas do dia a dia. Também conta como ponto forte sua otimização de performance para manipulação de dados.

As funções do dplyr reproduzem as principais tarefas da manipulação, de forma bastante intuitiva.



O **select()** é a função mais simples de ser entendida. É usada para selecionar variáveis (colunas, campos, features...) do seu data frame. Pode ser feito também uma “seleção negativa”, ou seja, escolher as colunas que não quer.

Exemplo 18.A:

```
library(dplyr)  
dados.1<-dados%>%select(Idade, Sexo)  
head(dados.1)
```

Exemplo 18.B:

```
library(dplyr)  
dados.1<-dados%>%select(-Idade, -Sexo)  
head(dados.1)
```



Além de escolher apenas alguns campos, você pode escolher apenas algumas linhas, utilizando alguma condição como filtragem. Para isso, basta utilizar a função **filter**. Exemplo 19:

```
dados1 <- dados %>%  
  select(Idade, Peso, Sexo, Renda) %>%  
  filter(Sexo == "1" & Renda=="3")  
dados1
```

Para criar novos campos, podemos usar o **mutate**. Exemplo 20:

```
dados1 <- dados %>%  
  select(Idade, Peso, Sexo, Renda) %>%  
  mutate(Sexo == "1" & Renda=="3")  
dados1
```



Group By e Summarise

O **group_by()** e o **summarise()** são operações que trabalham na agregação dos dados, ou seja, um dado mais detalhado passa a ser um dado mais agregado e agrupado, em consequência disso, menos detalhado. O agrupamento de dados geralmente é trabalhado em conjunção com sumarizações, que usam funções matemáticas do tipo soma, média, desvio padrão etc. Enquanto o **group_by()** “separa” seus dados nos grupos que você selecionar, o **summarise()** faz operações de agregação de linhas limitadas a esse grupo.

Exemplo 21:

```
install.packages("nycflights13")  
library(nycflights13)  
data("flights")  
str(flights)
```

Vamos obter a média de atraso da chegada e da partida para cada mês. Para isso, primeiro agrupamos no nível necessário e depois sumarmos. Exemplo 22:

```
media <- flights %>%  
  group_by(month) %>%  
  summarise(arr_delay_media = mean(arr_delay, na.rm=TRUE),  
    dep_delay_media = mean(dep_delay, na.rm=TRUE))  
media
```

O **operador** `%>%` é usado para encadeamento das manipulações, ou seja, informa que um resultado da operação anterior será a entrada para a nova operação.



A função **arrange()** serve para organizar os dados em sua ordenação. Costuma ser uma das últimas operações, normalmente usada para organizar os dados e facilitar visualizações ou criação de relatórios. Utilizando o exemplo anterior, gostaríamos de ordenar os meses pelas menores médias de decolagem (para ordens decrescentes basta usar o sinal de menos -). Exemplo 23:

```
media <- flights %>%  
  group_by(month) %>%  
  summarise(arr_delay_media = mean(arr_delay, na.rm=TRUE),  
    dep_delay_media = mean(dep_delay, na.rm=TRUE)) %>%  
  arrange(dep_delay_media)  
media
```



*Obrigada pela
participação.*